
Go Learning Journey

Phase 2 — Concurrency & Standard Library

■ 2–3 Weeks Duration	■ 3 Projects Hands-on	■ 10 Topics Covered	■ Race Detector From Day 1
--------------------------------	---------------------------------	-------------------------------	--------------------------------------

Prerequisites: Phase 1 complete — fundamentals, error handling, modules, receiver types, interface patterns, and both Phase 1 projects.

This phase covers Go's defining feature — the goroutine + channel concurrency model — alongside the standard library packages every production Go service depends on.

Table of Contents

01 Mental Model First	Goroutines vs threads, the scheduler, why it matters
02 Week 1 — Goroutines & Channels	go keyword, channel mechanics, buffered vs unbuffered, select
03 Week 2 — Sync, Context & stdlib I/O	WaitGroup, Mutex, context.Context, net/http, encoding/json, os/io/bufio
04 Exercises & Drills	13 progressive exercises with goals and gotchas
05 Project 1 — Port Scanner	Worker pool, fan-out, WaitGroup, net.DialTimeout
06 Project 2 — Weather CLI	HTTP client, JSON decoding, flag, env vars
07 Project 3 — Concurrent File Downloader	Fan-out, result collection, io.Copy, atomic counters
08 Patterns Reference	Done channel, pipeline, fan-out/fan-in, errgroup
09 Common Mistakes & Gotchas	Leaks, race conditions, nil channels, context misuse
10 Resources & What's Next	Canonical reads, Phase 3 preview

Mental Model First

Before you write a single goroutine, get this locked in. The concurrency model is not a feature bolted onto Go — it is a core design decision that shapes everything from the runtime scheduler to how you structure production services.

Goroutine vs OS Thread

Property	OS Thread	
Initial stack	~1–8 MB fixed	~2 KB, grows as needed
Creation cost	Heavy (syscall)	Extremely cheap
Scheduling	OS kernel	Go runtime (M:N scheduler)
Context switch	Expensive (~1–10 μs)	Cheap (cooperative)
Practical limit	Hundreds to low thousands	Tens of thousands+
Blocked I/O	Blocks the OS thread	Parks goroutine, thread continues
Communication	Shared memory + locks	Channels (preferred) + sync

The M:N Scheduler

Go runs goroutines on a pool of OS threads using an **M:N scheduler** (M goroutines on N OS threads). The key components are:

- **G — Goroutine** — The unit of concurrent execution. Extremely lightweight.
- **M — Machine** — An OS thread. Managed by the Go runtime, not the OS scheduler.
- **P — Processor** — A logical processor. Each P has a run queue of goroutines. GOMAXPROCS controls how many Ps exist (default: number of CPU cores).

■ **The key insight from Node.js:** In Node, async is concurrent but single-threaded — one event loop. In Go, goroutines can run in **true parallel** across multiple OS threads. This means shared mutable state accessed from two goroutines **will** race unless protected. This is why the race detector must be on from day one.

Race Detector — Use It Always

Run every concurrent program with the race detector enabled:

```
go run -race main.go
go test -race ./...
go build -race -o myapp
```

■ The race detector adds ~5–10× overhead — use it during development and CI, not in production binaries.

02

Week 1 — Goroutines, Channels, select

Goroutines

Launching a goroutine is one keyword: `go`. That's the syntax. The substance is everything that happens after.

```
func worker(id int) {
    fmt.Printf("worker %d starting\n", id)
    // simulate work
    time.Sleep(time.Second)
    fmt.Printf("worker %d done\n", id)
}

func main() {
    for i := 1; i <= 5; i++ {
        go worker(i) // launch goroutine
    }
    // BUG: main exits before workers finish!
    // Fix: use sync.WaitGroup (see Week 2)
}
```

- **Goroutines are not threads** — You can launch 10,000 and the runtime manages them across available OS threads.
- **Arguments are evaluated immediately** — The value passed to `go f(x)` is captured at the `go` statement, not when the goroutine starts.
- **Loop variable capture is a classic bug** — In a for-loop, `go func() {{ use(i) }}()` captures the loop variable by reference. Copy it: `go func(n int) {{ use(n) }}(i)`.
- **No goroutine IDs** — Intentional. Go does not expose goroutine IDs. Do not try to work around this.

Channels

Channels are typed conduits for communication between goroutines. They are first-class values — you can pass them as arguments, store them in structs, and range over them.

Unbuffered channel `ch := make(chan int)`

Sender blocks until receiver is ready, and vice versa. Guarantees synchronisation at the point of exchange. Use for coordination.

Buffered channel `ch := make(chan int, 10)`

Sender blocks only when buffer is full. Receiver blocks only when buffer is empty. Use as a work queue or to decouple producer/consumer speed.

Directional channels `chan<- int / <-chan int`

Pass channels with restricted direction to functions — prevents bugs and documents intent at the type level.

Closing a channel `close(ch)`

Signals no more values will be sent. Receivers see remaining values then zero-values. **Never close from the receiver side. Never close twice.**

Range over channel `for v := range ch { }`

Reads until channel is closed. This is the idiomatic way to drain a results channel.

Non-blocking receive `v, ok := <-ch`

`ok` is `false` if channel is closed and empty. Used to distinguish zero-value from closed.

select Statement

The `select` statement is to channels what `switch` is to values. It blocks until one of its cases can proceed, then executes it. If multiple cases are ready simultaneously, one is chosen **at random**.

- **Timeout on an operation** — `case <-time.After(2*time.Second):` — proceed if nothing arrives within 2 s
- **Non-blocking send/receive** — Add `default:` case — executes immediately if no other case is ready
- **Done channel / cancellation** — `case <-done:` — the idiomatic way to signal goroutine shutdown
- **Multiplex N channels** — Select across multiple input channels — whichever has data first wins
- **Nil channel blocks forever** — A `nil` channel in a select case is never selected — use this to disable cases dynamically

```
func process(ch <-chan int, done <-chan struct{}) {
    for {
        select {
        case v, ok := <-ch:
            if !ok { return } // channel closed
            fmt.Println("got:", v)
        case <-done:
            fmt.Println("cancelled")
            return
        case <-time.After(5 * time.Second):
            fmt.Println("timeout – no data for 5s")
        }
    }
}
```

Week 2 — Sync, Context & stdlib I/O

sync Package

`sync.WaitGroup` — Coordinate shutdown of N goroutines.

- Call `wg.Add(n)` before launching, `wg.Done()` in each goroutine (via `defer`), `wg.Wait()` to block until all finish.
- Never copy a `WaitGroup` — always pass by pointer or use it as a struct field.
- Never call `wg.Add()` inside the goroutine itself — race between `Add` and `Wait`.

`sync.Mutex` — Protect shared state from concurrent access.

- Lock with `mu.Lock()`; always unlock with `defer mu.Unlock()` immediately after.
- Use `sync.RWMutex` when reads vastly outnumber writes — allows concurrent readers.
- Keep the critical section small. Never call external functions while holding a lock.
- Prefer channels for **communication**; use `Mutex` for **protecting state** (counters, maps, caches).

`sync.Once` — Run initialisation code exactly once, safely across goroutines.

- `once.Do(fn)` — `fn` is called the first time only, even if 100 goroutines call `Do` simultaneously.
- Classic use: lazy singleton initialisation (DB connection, config load).

`sync.Map` — A concurrent-safe map for specific access patterns.

- Use only when keys are written once and read many times, or when goroutines work on disjoint key sets.
- For most use cases, a regular map + `sync.RWMutex` is clearer and equally fast.

`atomic` — Lock-free operations on single integer/pointer values.

- Package `sync/atomic` — `atomic.AddInt64`, `atomic.LoadInt64`, `atomic.StoreInt64`.
- Use for single-value counters (progress tracking, request counts). Not a replacement for `Mutex` on structs.

context Package

Every function that does I/O, calls another service, or runs as part of a request should accept `ctx context.Context` as its **first parameter**. This is not a style preference — it is the standard contract in Go.

<code>context.Background()</code>	Root context. Use at the top of main or in tests. Never inside request handlers.
<code>context.TODO()</code>	Placeholder when you know a context should be added but haven't wired it yet.
<code>context.WithCancel(ctx)</code>	Returns a child context + cancel func. Call <code>cancel()</code> when done — always, via <code>defer</code> .
<code>context.WithTimeout(ctx, d)</code>	Auto-cancels after duration <code>d</code> . Returns child <code>ctx</code> + cancel func (still call it!).
<code>context.WithDeadline(ctx, t)</code>	Like <code>WithTimeout</code> but with an absolute time.
<code>context.WithValue(ctx, k, v)</code>	Attach request-scoped values (trace IDs, auth). Not for optional parameters.

- Never store context in a struct — Pass it explicitly. Stored contexts silently outlive their intended scope.
- Always call `cancel()` — Even if the context expires naturally. Use `defer cancel()` immediately after creating it.
- Check `ctx.Err()` in loops — Long-running goroutines must check `ctx.Done()` or `ctx.Err()` to honour cancellation.
- `WithValue` is not a function parameter replacement — Only use it for cross-cutting concerns: trace IDs, request IDs, auth tokens.

net/http

The stdlib HTTP package is production-capable. Learn what it does before reaching for a framework — most of what frameworks add is routing sugar.

HTTP Server

- `http.HandleFunc("/path", handlerFunc)` — register a handler
- `http.ListenAndServe(":8080", nil)` — start server with default mux
- Handler signature: `func(w http.ResponseWriter, r *http.Request)`
- Always check `r.Context()` for cancellation in handlers that do I/O
- `http.Error(w, msg, code)` for error responses — do not forget to return after calling it

HTTP Client

- `http.Get(url)` — quick GET, but use a configured `http.Client` in production
- Always set `Timeout` on your `http.Client`: `&http.Client{Timeout: 10*time.Second}`
- Always defer `resp.Body.Close()` — even if you don't read the body
- `http.NewRequestWithContext(ctx, method, url, body)` — the correct way to make cancellable requests
- Do not use the package-level `http.Get` in production — it has no timeout

encoding/json

JSON handling is struct-based. Define a struct with `json:"field_name"` tags and let the encoder/decoder do the mapping.

<code>json.Marshal(v)</code>	Struct → JSON bytes. Returns (<code>[]byte, error</code>).
<code>json.Unmarshal(data, &v;)</code>	JSON bytes → struct. Pass a pointer.
<code>json.NewEncoder(w).Encode(v)</code>	Write JSON to an <code>io.Writer</code> (<code>http.ResponseWriter</code> , file). Preferred for HTTP.
<code>json.NewDecoder(r).Decode(&v;)</code>	Read JSON from an <code>io.Reader</code> (request body). Preferred for HTTP.
<code>`json:"name"`</code>	Map struct field to JSON key name.
<code>`json:"name,omitempty"`</code>	Omit field from output if zero value.
<code>`json:"- "`</code>	Always omit this field.

os, io, bufio

<code>os.Open</code> / <code>os.Create</code>	Open for reading / create for writing. Always defer <code>Close()</code> .
<code>os.ReadFile</code> / <code>os.WriteFile</code>	Convenience for small files. Reads/writes entire content in one call.
<code>os.Getenv("KEY")</code>	Read environment variable. Returns empty string if not set.
<code>io.Copy(dst, src)</code>	Stream data from <code>src</code> to <code>dst</code> without loading into memory. Use for large files and HTTP response bodies.
<code>io.ReadAll(r)</code>	Read entire reader into <code>[]byte</code> . Convenient but loads all into memory — avoid for large payloads.
<code>bufio.NewReader(r)</code>	Buffered reader. Use for line-by-line reading from files or stdin.

04

Exercises & Drills

Work through these in order. Each exercise is small enough to finish in 30–90 minutes but targets exactly one concept. Read the gotcha before you start — it will save you an hour.

E-01 Hello Goroutine

Week 1

Launch 5 goroutines that each print their ID. Add a `time.Sleep` in main to let them finish. Then remove the `Sleep` — observe what happens.

Concepts: `channel` / `WaitGroup`

■ Gotcha: `main()` exits the moment it returns, killing all goroutines. `Sleep` is not the fix — it teaches you WHY you need `WaitGroup`.

E-02 WaitGroup Shutdown

Week 1

Rewrite E-01 using `sync.WaitGroup`. Verify that all goroutines complete before main exits regardless of how long they take.

Concepts: `sync.WaitGroup`

■ Gotcha: Call `wg.Add()` before launching, not inside the goroutine. `wg.Done()` via `defer`.

E-03 Unbuffered Pipeline

Week 1

Build a two-stage pipeline: generator goroutine sends ints 1–10 on a channel; processor goroutine squares them and prints. Use `close()` to signal completion.

Concepts: `unbuffered channel`, `close`, `range`

■ Gotcha: `Range` over a channel blocks until it is closed. Don't forget `close()` or it deadlocks.

E-04 Buffered Work Queue

Week 1

Create a buffered channel of size 5. Send 20 jobs from main. Launch 3 worker goroutines that receive and process jobs. Measure time vs sequential.

Concepts: `buffered channel`, `worker pool`

■ Gotcha: Buffered channels are not magic parallelism — they decouple producer and consumer speed.

E-05 select + timeout

Week 1

Write a function that reads from a channel with a 500ms timeout using `select`. If data arrives in time, print it. If not, print 'timeout'.

Concepts: `select`, `time.After`

■ Gotcha: `time.After` creates a new timer channel each iteration in a loop — use `time.NewTimer` for loops to avoid goroutine leaks.

E-06 Done Channel Pattern

Week 1

Create a goroutine that runs indefinitely in a loop. Signal it to stop using a done channel. Verify it actually stops.

Concepts: `done`, `channel`, `select`

■ Gotcha: This is the foundation of context cancellation. Learn the manual version first.

E-07 Race Condition Hunt

Week 1

Write code with a deliberate race condition (two goroutines incrementing a shared counter). Run with `-race` to see it caught. Fix with `atomic`, then with `Mutex`.

Concepts: `go run -race`, `atomic`, `Mutex`

■ Gotcha: The race detector only catches races that actually execute — it cannot prove race-freedom.

E-08 Mutex vs RWMutex

Week 2

Build a thread-safe in-memory cache (`map[string]string`). Implement with `Mutex` first, then benchmark with `RWMutex` when reads $\gg$ writes.

Concepts: `sync.Mutex`, `sync.RWMutex`

■ Gotcha: `RWMutex` has higher overhead than `Mutex` for write-heavy workloads. Benchmark before assuming it's faster.

E-09 Context Cancellation

Week 2

Write a function that does 'work' in a loop (`time.Sleep`). Accept `ctx context.Context` and check `ctx.Done()` each iteration. Cancel it from main after 2 seconds.

Concepts: `context.WithCancel`, `ctx.Done()`

■ Gotcha: `ctx.Done()` returns a channel — select on it alongside your work channel.

E-10 Context Timeout

Week 2

Make an HTTP GET request using `http.NewRequestWithContext` with a 1-second timeout. Try against a slow endpoint (`httpbin.org/delay/3`). Handle the timeout error.

Concepts: `context.WithTimeout`, `http.Client`

■ Gotcha: Always check if the error is a context error: `errors.Is(err, context.DeadlineExceeded)`.

E-11 JSON Round-trip

Week 2

Define a struct for a GitHub repo (`name`, `stars`, `language`, `created_at`). Marshal to JSON, write to a file, read the file back, Unmarshal, and print the result.

Concepts: `encoding/json`, `os`, `struct tags`

■ Gotcha: `time.Time` marshals to RFC 3339 by default. Unmarshal requires the same format — use `json.RawMessage` for flexible parsing.

E-12 HTTP Server + JSON

Week 2

Write an HTTP server with two routes: GET /health (returns {"ok": true}) and GET /echo that reads a JSON body and returns it. Use `json.NewDecoder` and `json.NewEncoder`.

Concepts: `net/http`, `encoding/json`

■ Gotcha: Always return after `http.Error()` — it does not stop handler execution. Close the request body.

E-13 File Line Counter

Week 2

Read a large text file line-by-line using `bufio.Scanner`. Count lines, words, and bytes without loading the file into memory. Print a `wc`-style summary.

Concepts: `bufio.Scanner`, `os.Open`

■ Gotcha: Scanner has a default 64KB line buffer. For very long lines use `scanner.Buffer()` to increase it.

05 / 06 / 07

Projects

Build the projects in order. Each one is a complete program, not a toy. The port scanner is mandatory before anything else — do not skip it.

■ Project 1 — Port Scanner

Difficulty	■■■	Timeline	3–4 days	Why first	Visceral concurrency aha-moment
------------	-----	----------	----------	-----------	---------------------------------

A sequential port scanner checking ports 1–1024 takes minutes. A goroutine-based version with a worker pool completes in seconds. Run both and measure the difference — it permanently changes how you think about I/O-bound work.

Phase 1: Sequential baseline

- Use `net.DialTimeout` to probe a single port
- Loop over ports 1–1024, measure total time
- This is your benchmark — write it down

Phase 2: Goroutine per port (naive)

- Launch a goroutine for each port with `go`
- Collect results via a channel, use `sync.WaitGroup` to know when done
- Run with `go run -race` — fix any races
- Measure time — it should be dramatically faster

Phase 3: Worker pool (correct)

- Create a buffered channel as a work queue: `jobs := make(chan int, 100)`
- Launch N workers (try N=100) that read from jobs and send results to a results channel
- Feed jobs from main, close jobs when done, range over results
- This bounds concurrency — essential for production code

Phase 4: Context + timeout

- Add a root context with timeout: `context.WithTimeout(context.Background(), 30*time.Second)`
- Pass ctx to each dial operation so the whole scan respects the deadline
- Handle `context.DeadlineExceeded` gracefully

Phase 5: Output

- Sort open ports before printing (results arrive out of order)
- Add a `--timeout` flag, `--ports` flag (range like 1-1024 or list like 22,80,443)
- Print closed ports only in verbose mode

You will learn: `net.DialTimeout` · `sync.WaitGroup` · worker pool pattern · buffered channels as queues · `context` propagation · `flag` package · result aggregation

■ Project 2 — Weather CLI

Every real backend service makes HTTP calls and decodes JSON. This project teaches Go's HTTP client model, real-world error handling, and the patterns you will use in every production service you write.

Phase 1: Basic HTTP GET

- Use `http.Get` to call the OpenWeatherMap API (free tier)
- Print the raw response body with `io.ReadAll`
- Always `defer resp.Body.Close()` — remember this forever

Phase 2: JSON decoding

- Define structs for the API response with json struct tags
- Use `json.NewDecoder(resp.Body).Decode(&weather;)`
- Print a formatted summary: city, temp, description, humidity

Phase 3: Proper HTTP client

- Replace `http.Get` with a configured `http.Client` with timeout
- Use `http.NewRequestWithContext` so the request respects context cancellation
- Handle non-200 status codes as errors

Phase 4: CLI flags + env

- Use the `flag` package: `--city`, `--units` (metric/imperial), `--format` (text/json)
- Read API key from `os.Getenv("OPENWEATHER_API_KEY")` — never hardcode it
- Print a usage message if the key is missing

Phase 5: Error handling

- Distinguish network errors, API errors (bad key, city not found), and JSON errors
- Return meaningful error messages — not just 'error occurred'
- Handle rate limiting (HTTP 429) with a helpful message

You will learn: `http.Client` · `http.NewRequestWithContext` · `json.NewDecoder` · struct tags · `flag` package · `os.Getenv` · HTTP status handling · `defer resp.Body.Close()`

■ Project 3 — Concurrent File Downloader

Download from multiple URLs simultaneously, write to disk concurrently, aggregate results, and track progress. This is the complete fan-out/fan-in pattern you will see in every real service.

Phase 1: Sequential baseline

- Accept a list of URLs (from a file or CLI args)
- Download each sequentially with `http.Get` and `io.Copy` to a file
- Measure total time — this is your baseline

Phase 2: Concurrent fan-out

- Launch one goroutine per URL, use `sync.WaitGroup` to wait for all
- Collect results (filename, size, error) via a results channel
- Handle the case where some downloads fail — don't panic, aggregate errors

Phase 3: Worker pool

- Bound concurrency: N workers (default 5, configurable) reading from a jobs channel
- This prevents opening 1000 connections simultaneously on a large URL list
- Results channel size = number of jobs (buffered)

Phase 4: Progress tracking

- Use `sync/atomic` to track bytes downloaded across goroutines
- Print progress updates from a separate goroutine using a ticker: `time.NewTicker`

- This goroutine also needs to respect the done channel / context

Phase 5: Robustness

- Add context with timeout for the whole operation: `context.WithTimeout`
- Retry failed downloads once before marking as error
- Write a summary at the end: N succeeded, M failed, total bytes, time taken

You will learn: `sync.WaitGroup` · result channels · `io.Copy` streaming · `sync/atomic` counters · `time.NewTicker` · error aggregation · fan-out/fan-in · worker pool · context propagation

08

Patterns Reference

These are the canonical Go concurrency patterns. You will use all of them.

Done Channel

Signal goroutines to stop via a channel. The predecessor to `context.Context`.

- Create `done := make(chan struct{})`
- Select on `<-done` in the goroutine's main loop
- `close(done)` to broadcast shutdown to all goroutines reading it
- `close()` broadcasts; sending a value wakes only one goroutine

Pipeline

Chain goroutines where output of one is input to the next. Each stage is a goroutine.

- Stage 1 generates values → channel → Stage 2 transforms → channel → Stage 3 outputs
- Each stage closes its output channel when done
- Pass done channel through all stages for cancellation
- Use for streaming data transformations (parse → validate → enrich → store)

Fan-out

Distribute work from one channel to multiple goroutines.

- One input channel, N workers all reading from it
- Workers compete for jobs — first available worker gets the next job
- This is the worker pool pattern — bound N to control concurrency
- Used in port scanner, file downloader, batch processors

Fan-in

Merge results from multiple goroutines into a single channel.

- N goroutines each have their own output channel
- A merge goroutine reads from all N and sends to a single results channel
- Use `sync.WaitGroup` inside merge goroutine to know when to close results
- Alternative: use `errgroup` from golang.org/x/sync

errgroup

golang.org/x/sync/errgroup — the ergonomic `WaitGroup` for goroutines that return errors.

- `g, ctx := errgroup.WithContext(parentCtx)`

- `g.Go(func() error { ... })` — launch goroutine, error cancels the group `ctx`
- `err := g.Wait()` — first non-nil error is returned
- Cancels all goroutines when any one fails — context propagation for free

Semaphore

Limit the number of goroutines doing something at the same time.

- `sem := make(chan struct{}, N)` — buffered channel as semaphore
- `sem <- struct{}{}` — acquire (blocks when N goroutines are active)
- `<-sem` — release (always via `defer`)
- Or use golang.org/x/sync/semaphore for a more ergonomic API

09

Common Mistakes & Gotchas

These will happen to you. Read them now so they happen less.

Goroutine leak CRITICAL

A goroutine blocked on a channel send/receive with nobody on the other end runs forever. Always ensure every goroutine has a path to exit. Use context cancellation or a done channel. `goleak` (github.com/uber-go/goleak) detects leaks in tests.

Loop variable capture VERY COMMON

In Go < 1.22: `for i := range items {{ go func() {{ use(i) }}() }}` captures the loop variable by reference — all goroutines see the last value. Fix: `go func(v Item) {{ use(v) }}(item)` or `item := item` before the `go` statement. Go 1.22+ fixed this — each iteration gets its own variable.

Sending on a closed channel PANIC

Sending to a closed channel panics. Only the sender should close a channel. If multiple goroutines send, use a `sync.Once` or a dedicated closer goroutine.

Nil channel blocks forever SILENT DEADLOCK

Reading or writing to a nil channel blocks forever. Selecting on a nil channel case means that case is never selected. This can be useful (disabling a select case dynamically) but usually indicates a bug.

Forgetting defer cancel() GOROUTINE LEAK

Every `context.WithCancel` / `WithContext` / `WithDeadline` returns a cancel function. If you don't call it, the parent context's resources are leaked until the parent is cancelled. Always: `ctx, cancel := context.WithTimeout(...); defer cancel()`

Not checking resp.Body.Close() CONNECTION LEAK

HTTP response bodies must be closed even if you don't read them. Always `defer resp.Body.Close()` immediately after checking the error from `http.Get` or `client.Do`.

Using http.Get in production TIMEOUT BUG

The package-level `http.Get` uses `http.DefaultClient` which has **no timeout**. Your program will hang forever on a slow or unreachable server. Always use a configured `http.Client` with a `Timeout` set.

Unlocking a Mutex in the wrong goroutine PANIC

A Mutex must be unlocked by the same goroutine that locked it. Use `defer mu.Unlock()` immediately after `mu.Lock()` to ensure this.

Race on map CRASH / DATA CORRUPTION

Concurrent reads and writes to a Go map without synchronisation cause a runtime crash (Go 1.6+). Use `sync.RWMutex` around all map access, or use `sync.Map` for specific access patterns.

WaitGroup.Add() inside goroutine RACE

If you call `wg.Add(1)` inside the goroutine you're launching, main may call `wg.Wait()` before the `Add` happens. Always call `wg.Add()` before the `go` statement in the launching goroutine.

10

Resources & What's Next

Canonical Resources for Phase 2

go.dev/tour	Primary reference
Concurrency section	
Interactive. Do the concurrency section first — goroutines through select. Required.	
Rob Pike — Go Concurrency Patterns (2012)	Watch once, keep slides
talks.golang.org/2012/concurrency.slide	
Canonical. Pipeline, fan-out, done channel, timeout — all here. Required.	
Go Blog — Share Memory By Communicating	30 min read
go.dev/blog/codelab-share	
The philosophical foundation. Explains WHY channels, not just how.	
Go Blog — Go Concurrency Patterns: Context	30 min read
go.dev/blog/context	
The official guide to context.Context. Read before touching net/http.	
100 Go Mistakes — Ch. 8 & 9	Reference
Teiva Harsanyi	
Read chapters 8 (goroutines/channels) and 9 (standard lib) alongside projects. High signal.	
Alex Edwards — alexedwards.net	net/http deep dive
Let's Go blog posts	
Best practical guide to Go's HTTP layer. Free blog posts cover everything for the Weather CLI.	

Phase 3 Preview — What's Coming

Testing	Table-driven tests, subtests, testify, mocking, integration tests
Databases	database/sql, pgx, GORM, migrations, connection pooling
REST APIs	Router (chi / gin), middleware chain, request validation, error handling
gRPC	Protobuf, service definitions, streaming, interceptors

Observability	Structured logging (slog), metrics (prometheus), distributed tracing
CLI tooling	cobra, viper, config management

Phase 2 completion checkpoint: You have built the port scanner, weather CLI, and concurrent file downloader. You can explain goroutine leaks, choose between channels and mutexes, and propagate context through a call tree. You are ready for Phase 3.